# LIBXSTREAM Samples

## DBM — OpenCL

OpenCL backend of CP2K's DBM (Distributed Block-sparse Matrices). DBM processes batches of small matrix multiplications (tasks):

```
typedef struct {
  int m;
  int n;
  int k;
  int offset_a;
  int offset_b;
  int offset_c;
} dbm_task_t;
```

Each task is characterized by its M, N, and K and by offsets into compact arrays holding the A, B, and C matrices of an entire batch. A universal kernel covers a general range of shapes, and homogeneous batches are specialized at runtime. The entry point is `dbm_multiply_opencl_launch_kernel` in dbm_opencl.h.

### Build

```
cd samples/dbm
make
```

Produces `dbm_bench.x`, which is the benchmark driver of the SMM sample linked with the DBM backend. The SMM objects are built by the SMM sample, hence its make variables (`WITH_GPU`, `ELEM_TYPE`) apply. Requires an OpenCL runtime, BLAS, and LIBXS built from a sibling directory.

### Test

```
make test
```

Runs `dbm_bench.x` with `DBM_MULTIPLY_SMM=-1`, which routes the homogeneous LIBSMM batches into the DBM kernel and validates the result. The test fails if the DBM kernel did not run. The arguments of `dbm_bench.x` are those of `acc_bench.x` and can be given as `make test TESTARG="5 3000 13x5x7"`.

### CP2K

CP2K builds the backend into its own targets without naming its files: CMake calls `libxstream_add_dbm(<target>)` after `find_package(libxstream)`, and the Makefile in CP2K's `src/dbm` includes `dbm.mk` from this directory. Both generate `dbm_kernels.h` into the consumer's build tree. To build CP2K's DBM miniapp against this directory:

```
make miniapp [CP2K_ROOT=/path/to/cp2k]
```

`CP2K_ROOT` defaults to a `cp2k` directory next to LIBXSTREAM, and `dbm_miniapp.x` is built in `$CP2K_ROOT/src/dbm`. Unlike `make test`, the miniapp also exercises heterogeneous batches.

### Environment Variables

| Variable | Default | Description |
|---|---|---|
| DBM_MULTIPLY_SMM | 0 | Positive: homogeneous batches up to this size use LIBSMM (1 = 64) |
| | | Negative: LIBSMM (DBCSR) uses the DBM kernel for homogeneous |
| | | batches (backward usage, experimental) |
| DBM_MULTIPLY_SORT | 0 | Order of tasks in a batch: 0 as generated, 1 by shape |
| DBM_MULTIPLY_KERNEL | - | Path to a `.cl` file that replaces the embedded kernel |
| DBM_MULTIPLY_FP | 0 | 1: compute in single precision (data remains double precision) |
| DBM_MULTIPLY_BN | auto | Column tile (1–32), 8 or 2 on NVIDIA |
| DBM_MULTIPLY_BK | 0 | Caps the K-block (1, 4, 8, or 16 derived from the batch's max K) |
| DBM_MULTIPLY_SM | 0 | Non-zero: accumulate in shared local memory |
| DBM_MULTIPLY_WG | auto | Work-group size |
| DBM_MULTIPLY_LU | 0 | Unrolling: -2 full, -1 no hints, 0 inner, 1 outer-dehint |
| DBM_MULTIPLY_RO | -1 | Read-only operands: negative = auto, 0 = global, 1 = constant |
| DBM_MULTIPLY_XF | -1 | Intel 256-GRF mode: negative = device default, 0 = off, 1 = on |
| DBM_MULTIPLY_NZ | 0 | Non-zero:

skip zero contributions when accumulating into C | | DBM_MULTIPLY_LIN | 0 | Non-zero: swap the access pattern of A and B | | DBM_MULTIPLY_SGB | 1 | Sub-group broadcast on Intel GPUs (0 = off), opt-in on NVIDIA (1) | | DBM_MULTIPLY_BLK | auto | Row per lane for mixed shapes (1), also homogeneous (2), off (0) | | DBM_MULTIPLY_FUSE | auto | Tasks per work-group fusing those sharing A: 16 with BLK, else 1 |

`LIBXSTREAM_VERBOSE=2` (or higher) prints the kernel configuration and the compilation of each specialization, and values above 2 trace every launch.

## Ozaki Scheme — OpenCL

High-precision GEMM on OpenCL devices via mantissa slicing (Scheme 1) or Chinese Remainder Theorem (Scheme 2). Both schemes decompose FP matrices into int8/u8 tiles and use DPAS/XMX matrix engines when available. This is an OpenCL adaptation of the CPU-based Ozaki sample in LIBXS.

### Build

```
cd samples/ozaki
make [DBG=1]
```

Requires an OpenCL runtime and headers. BLAS is linked via `BLAS=2` for the reference GEMM.

### Run

```
./ozaki.x [M [N [K [transa [transb [alpha [beta [lda [ldb [ldc]]]]]]]]]]
```

All arguments are positional and optional:

| Pos. | Argument | Default | Description | ||||| | 1 | M | 257 | Rows of C and op(A) | | 2 | N | M | Columns of C and op(B) | | 3 | K | M | Inner dimension | | 4 | transa | 0 | 0=N, 1=T for A | | 5 | transb | 0 | 0=N, 1=T for B | | 6 | alpha | 1 | Scalar multiplier for A*B | | 7 | beta | 1 | Scalar multiplier for C | | 8 | lda | auto | Leading dimension of A | | 9 | ldb | auto | Leading dimension of B | | 10 | ldc | M | Leading dimension of C |

### Environment Variables

**Scheme Selection** | Variable | Default | Description | |||| | OZAKI | 3 | 1=mantissa slicing, 2=CRT, 3=adaptive (default when unset) | | OZAKI_FP | 64 | 64=fp64 (double), 32=fp32 (float) | | OZAKI_N | (auto) | Slices (Sch.1: fp64=8, fp32=4) or primes (Sch.2: fp64=16, fp32=9) | | OZAKI_XOVER | 27 | Sch.1/2 crossover weight used by OZAKI=3 (fp64, non-NVIDIA) |

OZAKI=3 (adaptive) compares the two costs per call and picks the cheaper path: the Scheme-1 pair count against the Scheme-2 prime count plus its reconstruction, weighted by `OZAKI_XOVER` and amortized over K. The comparison is stateless — it uses the static slice cutoff, not a measured one — so mixed matrix sizes in one process cannot influence each other. On NVIDIA it selects Scheme 2 outright, and in fp32 on other devices Scheme 1, both because counting GEMMs mispredicts there.

**Accuracy** | Variable | Default | Description | |||| | OZAKI_FLAGS | 3 | Sch.1 bitmask: 1=Triangular, 2=Symmetrize, 0=full S^2. No Sch.2 | | OZAKI_TRIM | 0 | Levels to trim (0=default). ~7 bits (Sch.1), ~4 bits (Sch.2); negative buys precision back | | OZAKI_SYMRES | (auto) | Sch.2: symmetric residues (magnitude m/2 at most). On with bf16, or K past 32768 | | OZAKI_GROUPS | (auto) | Sch.2: K-grouping factor (1=off). Consecutive K panels share reconstr. | | OZAKI_FRACCRT | (auto) | Sch.2: 0=Garner, 2=fractional CRT. Auto: 0 if unfused, else 2 |

`OZAKI_FLAGS` selects how the slice-pair loop is traversed, and the two bits mean the same on the host and on the device: 1 starts the inner slice at the outer one, 2 adds the transposed product of every off-diagonal pair. Only 0 (the full square loop) and 3 (both bits) compute the whole sum – 1 alone omits the transposed products and 2 alone counts them twice, so both are for symmetric operands only.

`OZAKI_GROUPS` splits K into panels to bound the preprocessing footprint, and it is a slowdown at every size measured because it is incompatible with the unfused epilogue (`OZAKI_UNFUSE`, on by default for GPUs) – set it only when the per-pass buffers do not fit. Left unset it is derived instead, and only where it is needed for a correct result: an

OZAKI_MAXK past what an integer accumulator holds exactly (32768 unsigned, 131072 symmetric) gets the largest panel that stays inside it. Raising OZAKI_MAXK is what turns it on, so expect the 2x there and set OZAKI_SYMRES=1 first if the K is within 131072. The bf16 carrier never needs it, narrower accumulator notwithstanding: the kernel folds that accumulator back to residues in place as it goes, which costs 16% of the GEMM and nothing end-to-end. OZAKI_GROUPS=1 declines the derived grouping and warns where that leaves K unreduced; OZAKI_GROUPS=0 is not that request, because it is what an unset knob reads as.

OZAKI_FRACCRT=1 trades exactness for speed (flat fractional sum, magnitude-bounded); modes 0 and 2 are both exact and differ only in speed. Which one is faster depends on the epilogue rather than on the device: with OZAKI_UNFUSE the reconstruction is a separate pass and Garner wins, without it the fractional variant does, so the default follows that knob.

**Hardware Control** | Variable | Default | Description | |||| | OZAKI_TM | (auto) | Output tile M (BM), with OZAKI_TN. Not under wgmma (below) | | OZAKI_TN | (auto) | Output tile N (BN), with OZAKI_TM. Not under wgmma (below) | | OZAKI_RTM | (auto) | Register tiling M (power of two). Auto: 2 (HIER), 4 (256-GRF) | | OZAKI_RTN | (auto) | Register tiling N. Auto: 8 (NV MMA Sch.2), 2 (Intel), 1 (other) | | OZAKI_SB | 1 | Sch.1: slice-block width for the pair loop (needs 256-GRF) | | OZAKI_WGS_MAX | (auto) | Work-group size ceiling for tile selection (0=hardware bound) | | OZAKI_LU | 0 | Unroll hint for the K-loop (0=compiler default) | | OZAKI_SKIP_GARNER| 0 | Sch.2: skip reconstruction (timing only, result is not a GEMM) | | OZAKI_WG | 0 | Work-group size hint (0=no hint) | | OZAKI_SG | (auto) | Sub-group size (forced to 16 with XMX) | | OZAKI_BIGGRF | (auto) | Override 256-GRF detection (0=off, 1=on). HIER defaults to 128 | | OZAKI_KU | 2 | K-loop unroll factor (16 with OZAKI_WGMMA, 8 if RS is off) | | OZAKI_RC | 8 | DPAS repeat count (8 or 4) | | OZAKI_PB | 1 | Sch.2: CRT prime batching factor | | OZAKI_HIER | (auto) | Sch.2: hierarchical CRT (default on). Two-level Garner reconstr. | | OZAKI_PREFETCH | 0 | Sch.1: enable prefetching | | OZAKI_SCALAR_ACC | 0 | Sch.1: force scalar accumulation | | OZAKI_BVNNI | 1 | NVIDIA: pre-interleave B so each operand is one aligned uint | | OZAKI_BKMAJOR | 0 | NVIDIA, Sch.2: transpose B to [N][K] instead (see below) | | OZAKI_WGMMA | (auto) | Sch.2: warp-group MMA. On where reachable (see below) | | OZAKI_WGMMA_N | 256 | Warp-group tile width, 64, 128 or 256 | | OZAKI_WGMMA_M | 128 | Warp-group tile rows: 128 = two warp groups, 64 = one | | OZAKI_WGMMA_RESIDE| (auto) | Sch.2: tiles per SM below which 128x128 runs two per SM (0=off) | | OZAKI_WGMMA_RESIDE_MKT | 2.5E8 | Sch.2: bound on M$K$tiles per SM for 128x128 (0=no bound) | | OZAKI_MAXNREG | 0 | Sch.2 wgmma: registers per thread of the GEMM (0=derived) | | OZAKI_NOBOUNDS | 0 | Sch.2: drop output range checks (whole-tile shapes only) | | OZAKI_ALPHA_ONE | 0 | Sch.2: specialize the GEMM for alpha=1 | | OZAKI_FIRST | 0 | Sch.2: specialize the GEMM for C=0+AB (first accumulation) | | OZAKI_UNFUSE | (auto) | Sch.2: reconstruct in a 2nd kernel. On for GPUs (see below) | | OZAKI_COMPLEX_3M | (auto) | Sch.2 complex: 1=3M on residues, 0=4M; auto: 3M from _MIN on | | OZAKI_COMPLEX_3M_MIN | (auto) | Cube root of M$N$K from which auto takes 3M (1024 coherent, 4096) | | OZAKI_BF16 | 0 | Sch.2: carry the residues in bf16 rather than int8 (see below) | | OZAKI_SWIZZLE | 0 | Sch.2: work-group rasterization width (0=launch order) | | OZAKI_TZDETECT | 0 | Sch.2: report the lossless OZAKI_TRIM the data allows (below) | | OZAKI_ARENA | (auto) | Device scratch arena in MB (0=off). Auto: on if no pool |

OZAKI_TZDETECT=1 reports how many low mantissa bits the operands actually have to spare, which is the largest OZAKI_TRIM that stays exact for that data. Operands holding integers, dyadic fractions or values promoted from a narrower type have bits to spare; a wide exponent range within a row spends them, because aligning the small elements shifts their spare bits out. The reported level is conservative, so OZAKI_TRIM up to it costs nothing in accuracy while every phase gets cheaper in proportion to the primes it saves. Reporting only: acting on it per call is not implemented.

On NVIDIA GPUs the Scheme-2 default RTN=8 is tuned for large K: it doubles the output tile per work-group, which needs a long K-loop to pay for itself (+38% at n=4096). Below K of about 1024 it costs 6% (K=512) to 21% (K=256) instead — set OZAKI_RTN=4 there.

Warp-group MMA runs the Scheme-2 GEMM on Hopper's wgmma instead of mma.sync. It is the default wherever it is reachable, and OZAKI_WGMMA=0 selects mma.sync. On an H100 PCIe it is faster at every size and bit-identical: the GEMM and reconstruction rows together are 13.66 -> 6.12 ms at n=4096 and 101.8 -> 45.8 at n=8192 against mma.sync at its own defaults, and transa, transb, rectangular and non-tile-multiple shapes gain the same. It needs hierarchical CRT, no K-grouping and OZAKI_PB=1, and it fixes the tile to OZAKI_WGMMA_MxOZAKI_WGMMA_N regardless of OZAKI_TM/OZAKI_TN.

"Reachable" is established by building a throwaway kernel during initialization rather than by inspecting the device, because nothing the device reports settles it: the advertised local-memory size is 48 KB on a part that accepts the 128 KB this path uses, and compute capability 9.0 covers hardware where the instruction no longer exists. If that

build fails, initialization says so under OZAKI_VERBOSE=1 and the mma.sync path is used with its own tuning, at no cost in speed.

Two knobs tune it, both set to the fastest measured value by default. OZAKI_KU sets how much K is staged per round (16 here); it is what the path spends shared memory on, 128 KB at the defaults, so lower it if a device refuses that. OZAKI_WGMMA_M=128 runs two warp groups per work-group instead of one, worth +4% at n=2048 and +18% at n=8192. Below OZAKI_WGMMA_RESIDE tiles per SM the tile narrows to 128x128 and two work-groups share an SM, which is faster there and slower above. The default is 8 on a part with a coherent host link and 2 on a discrete one, whose lower memory bandwidth moves the crossover down; OZAKI_WGMMA_RESIDE=0 keeps the wide tile. At long K the wide tile is kept once $MK$(tiles per SM) exceeds OZAKI_WGMMA_RESIDE_MKT, and OZAKI_WGMMA_RESIDE_MKT=0 lifts that bound.

Two implications worth knowing. The kernel is built twice — once from OpenCL C, then again from patched PTX — because warp-group MMA cannot be expressed in OpenCL C, so the first JIT of a shape costs about double. And if that patching fails the kernel is refused rather than run, since an unpatched kernel would silently compute zeros; OZAKI_VERBOSE=1 reports it.

OZAKI_UNFUSE=1 moves the CRT reconstruction out of the GEMM into a second kernel: the GEMM stores one residue byte per prime and output, gemm_crt_reduce reads them back and reconstructs C. It is the default on GPUs, OZAKI_UNFUSE=0 selects the fused epilogue, and both produce identical results.

This is faster because of what it removes rather than what it adds. With the prime loop outermost, the fused kernel must keep every output's group values live across it, which is 2 KB per work-item of dynamically indexed arrays — more than the L1 can hold for a work-group, so the epilogue runs at memory speed. A separate pass puts the output loop outermost and needs only four group values, i.e. registers. Measured at n=4096: 13.08 ms fused against 7.87 + 0.66 unfused (+53%), and the GEMM alone beats even its own loop-only time, because the frame was slowing the K-loop too. The gain is largest where the epilogue dominated: +158% at n=257, +80% at n=2048, +76% in fp32. It is not warp-group specific and helps mma.sync by 11%. Nor is it NVIDIA-specific: on a PVC (DPAS) the same figures are 0.66 -> 0.28 ms at n=257, 26.5 -> 22.0 at n=4096 and 15.8 -> 12.6 in fp32, also bit-identical.

The residue planes it needs are part of the per-call device scratch, together with the operand planes and the device copy of C. OZAKI_ARENA carves all of it from one allocation kept for the life of the context instead of creating and destroying it per call, which is worth a factor of 6.7 at n=4096 on a GH200 and 18% on an H100 PCIe. It is enabled by default wherever libxstream has no memory pool, so on NVIDIA nothing needs setting.

OZAKI_ARENA=0 restores per-call allocation. A positive value caps the arena in MB, and sizing it wrong is worse than not capping: a call needing more than the cap falls back per buffer, and a first call that exceeds it gets no arena at all. The requirement is roughly 1.2 GB at n=4096 in FP64 and scales with n^2, or call ozaki_scratch_size.

Set a positive value explicitly on a device that does have a pool but still wants the arena - LIBXSTREAM_USM on NVIDIA is that case, since the pool alone turns the arena off. Planes kept by OZAKI_CACHE are never carved, so the two knobs are independent. A caller that owns device memory can supply it instead with ozaki_scratch_size/ozaki_scratch_set (the benchmark does so under OZAKI_SCRATCH=1); the interceptor in LIBXS relies on the default, since a BLAS call cannot be handed a workspace.

Two things to know. Reading the profile, the two kernel rows have to be added for a total; the FLOP rate is attributed to the GEMM row alone. And it needs scratch memory of OZAKI_N bytes per output element, twice the size of C in fp64, which is freed per call.

OZAKI_SYMRES keeps each residue in [-m/2, m/2] rather than [0, m), which quarters the product magnitude and so quadruples the K an accumulator can hold exactly: 32768 to 131072 on int8, 256 to 1024 on bf16. It is therefore on by default with OZAKI_BF16, whose window is too small either way to spend, and on without it once OZAKI_MAXK exceeds 32768, where the signed reduction it costs is the cheaper of the two ways to stay exact – the other being OZAKI_GROUPS. Below that K an integer accumulator has the headroom anyway and the default stays unsigned. Either default can be overridden explicitly.

OZAKI_BF16=1 carries the CRT residues in bf16 instead of int8, so that Scheme 2 runs on a floating-point matrix engine. It needs warp-group MMA with the default A and B layouts, and it disables itself with a message otherwise. The moduli, the accuracy and the results are unchanged - the output is bit-identical to the int8 path - so it is a choice of engine and nothing else. Off by default: where an integer engine is available it is the faster one, by about 2.3x on the GEMM. Enable it on a device whose integer matrix throughput is much lower than its bf16 throughput.

OZAKI_TM and OZAKI_TN take effect only together – either one alone leaves the size-aware selection in charge – and neither applies where warp-group MMA runs, because that path's tile follows the geometry the instruction fixes and says so at OZAKI_VERBOSE=1. Within it the tile is still chosen per call from M and N.

`OZAKI_SWIZZLE=W` walks the tile grid in strips W tiles wide instead of in the launch order, so the work-groups resident at one time cover a block rather than a column of the grid and re-read the residue planes fewer times. It is off by default because it barely pays: measured at n=8192 it gains 4% on the GEMM (68.9 -> 66.1 ms at W=8) and costs the reconstruction pass 13% (2.51 -> 2.57), for about 4% net, and at n=4096 it is a wash. Worth trying only for large square problems.

`OZAKI_BKMAJOR=1` selects the transposed B layout, which exists for warp-group MMA (both operands K-major) and is exact like the default but slower for the kernels shipping today: at n=4096 it costs the fused GEMM 15.4 -> 21.0 ms and B preprocessing 0.5 -> 3.5 ms, because consecutive lanes then read `K_pad` apart instead of 4 bytes apart. It does not pay for the warp-group path either, although that one stages B cooperatively and could use 16-byte copies from a K-major layout: it measured 8.1 -> 9.5 ms at n=4096, because the loss in global-side coalescing outweighs the cheaper staging.

The hierarchical CRT groups primes for the two-level reconstruction, and the group size is derived from the prime count rather than fixed: 4 is the maximum the 32-bit level-2 datapath allows, but a group left holding a single prime is pathological. At the fp32 count of 9 primes, groups of 4 leave a 4,4,1 split whose reconstruction costs 1.21 ms at n=4096 - twice what a 12-prime run of three full groups costs - so fp32 uses groups of 3 and measures 0.83 ms. fp64 (16 primes) keeps groups of 4 and is unaffected. Regrouping changes the order of the final Horner summation, so an fp32 result can differ in the last bits from an earlier release, at the same accuracy (`linf_rel` around 1e-06 either way); integer CRT reconstruction is exact regardless of grouping.

No other setting differs by precision: the tile, staging depth, B layout and warp-group geometry were swept for fp32 and rank exactly as in fp64.

**Memory and Caching** | Variable | Default | Description | |||| | OZAKI_CACHE | 0 | Preprocessing cache bitmask: 1=A, 2=B, 3=both (or any negative) | | OZAKI_NPANEL | 1 | Sch.2: N-panel width (0=auto, 1=disable). See Panel Pipeline |

The preprocessing cache also stores the last effective cutoff from Scheme 1 occupancy detection. On cache hits the cutoff is reused without device-to-host readback, eliminating the sync bubble.

**Benchmark** | Variable | Default | Description | |||| | NREPEAT | 1 | Number of benchmark repetitions | | OZAKI_VERBOSE | 0 | 0=silent, 1=errors, 2=warnings, 3+=all. Neg.=all | | GRADE | 0 | Componentwise grade of every result (non-zero=on) | | EVIL | 0 | Per-column exponent span of 2n binades (n>0 only) | | TAME | 0 | Keep n mantissa bits of A and B, clear the rest | | XREF | 0 | Compensated reference on n rows (negative=all) |

`XREF=n` grades the Ozaki, the host BLAS and the cuBLAS result against a reference computed by `libxs_dot2_gemm` (as if in twice the double precision) on n rows spread over C, and prints one line `XREF: rows=n ozaki_linf_rel=.. blas_linf_rel=.. cubla` with `_l2_rel` and, under `GRADE`, `_grade` per result. The BLAS reference alone rounds as much as the results it grades, so it cannot show which of them is closer to the exact product.

Additional variables for accuracy monitoring and complex GEMM dispatch (OZAKI_THRESHOLD, OZAKI_STAT, OZAKI_EPS, OZAKI_RSQ, OZAKI_EXIT, OZAKI_COMPLEX) are handled by the LIBXS Ozaki sample (LIBXS), which owns the GEMM interceptor. See its README for details.

Kernel timings come from LIBXSTREAM rather than from a sample-specific facility: `LIBXSTREAM_PROFILE=1` reports one row per kernel (preprocess A, preprocess B, the fused GEMM, and so on), and `LIBXSTREAM_PROFILE_MEM=1` reports transfer rates. Neither needs a phase to be selected up front — every kernel is recorded and the interesting rows are read from the report.

**cuBLAS Reference** When the OpenCL headers are taken from a CUDA installation, the sample additionally links cuBLAS and reports a device-side reference GEMM (`cuBLAS GEMM` and `cuBLAS DIFF`). Build with `make CUBLAS=0` to opt out. The host BLAS stays the accuracy reference for both the Ozaki and the cuBLAS result, and a failing cuBLAS run is not fatal.

By default the sample requests FP64 emulation without the built-in fallback to native FP64 (CUDA 13.0u2 and later, compute capability 8.0 and later). The variables below are populated before cuBLAS is entered, hence any of them that is already set wins:

| Variable | Sample default | ||| | CUBLAS_EMULATE_DOUBLE_PRECISION | 1 | | CUBLAS_EMULATION_STRATEG
| eager | | CUBLAS_EMULATION_SPECIAL_VALUES_SUPPORT_MASK | 0 |

Setting `CUBLAS_EMULATE_DOUBLE_PRECISION=0` yields the native device GEMM, which is the baseline to compare against. For `OZAKI_FP=32` the counterpart is `CUBLAS_EMULATE_SINGLE_PRECISION`, which requires compute capability 10.0 and later.

| Variable | Default | Description | |||| | OZAKI_CUBLAS_BITS | 0 | Mantissa bits: 0=default, <0=match the OZAKI_N slices | | OZAKI_CUBLAS_XPTR | 0 | 1=pass LIBXSTREAM device pointers to cuBLAS (experiment) |

A non-zero `OZAKI_CUBLAS_BITS` fixes the number of int8 slices (`slices = ceil((bits + 1) / 8)`) instead of letting cuBLAS pick the precision per call. A negative value matches the component count of this sample, which is a slice count for OZAKI=1 but a prime count for OZAKI=2 — only the former is comparable.

`OZAKI_CUBLAS_XPTR=1` is an experiment and expected to fail: the device-side pointers of LIBXSTREAM are not addresses of the CUDA context that cuBLAS runs in.

Like the Ozaki timing, the cuBLAS timing covers the transfers, i.e. A, B and C are uploaded and C is read back per iteration; CUDA events report the device-side split (`gemm`, `h2d`, `d2h`) separately. With `OZAKI_CUBLAS_XPTR=1` the transfers are not on the CUDA timeline and read as zero.

The host buffers are pinned by OpenCL rather than by CUDA. LIBXSTREAM registers its host allocations with the CUDA runtime whenever the application links it, so no action is needed here; `LIBXSTREAM_VERBOSE=2` reports how many were accepted. `LIBXSTREAM_PIN=0` leaves them unregistered, which makes the cuBLAS transfers pageable — several times slower, and no longer comparable to the Ozaki row. `LIBXSTREAM_PIN=3` loads the CUDA runtime when the application did not link it, which is what a wrapper driver needs: it links `libOpenCL` alone, so the default setting finds no entry point and registers nothing.

Two properties to keep in mind when reading the numbers. The mode printed alongside the timing is the one that was requested: whether a call was emulated cannot be queried, and `nsys profile ./ozaki.x` is the way to confirm it. And emulation needs a workspace, which the sample supplies (`-DOZAKI_CUBLAS_WORKSPACE=<bytes>`, 2 GB by default, 0 to disable) because a workspace that is too small does not fail the call but silently falls back to non-emulated kernels.

### Output Tile Selection

The output tile per work-group (BM x BN) is chosen per call from M and N, because the best tile depends strongly on problem size: the total work-item count is invariant to the tile, so the tile only controls how many work-groups the problem splits into and how far M and N are padded to a tile multiple. A large tile maximizes operand reuse but produces too few work-groups to fill the device at small sizes; on a GPU Max 1550 the fixed 128x256 tile costs up to 1.7x at M=N=256..640 while being optimal from ~1024 upward.

Selection maximizes arithmetic intensity `BM*BN/(BM+BN)` per unit of padded work, subject to two constraints: the work-group size bound `SG * (BM/(XMX_M*RTM)) * (BN/(XMX_N*RTN)) <= max_wgs`, and enough work-groups to saturate the compute units (OZAKI_TILE_SAT). The intensity term is maximal for square tiles, which keeps the split balanced rather than degenerate. Set OZAKI_TM/OZAKI_TN to bypass selection and pin a tile.

The register tiling is selected per call as well, on Intel GPUs: the auto value in the table is the small-problem choice, and a larger one is taken once the problem still forms one tile per compute unit at the coarser granularity that implies. `OZAKI_VERBOSE=3` prints it as `rtm=2..4`, meaning either may be used, and the per-call value appears in the `JIT` line as `rt=`. Setting OZAKI_RTM or OZAKI_RTN pins both and bypasses this.

### Panel Pipeline (Scheme 2)

Only ~30% of a Scheme-2 call is the GEMM kernel; the rest is uploading A/B, preprocessing them into CRT residues, and downloading C. Scheme 2 therefore splits **N** into panels and pipelines them: while panel *j* runs its GEMM, panel *j+1* uploads and preprocesses its B columns and panel *j-1* downloads its C block. A is uploaded and preprocessed once as a prologue.

N is the right axis because each panel then owns a disjoint column block of B and C. C is read and written exactly once no matter how many panels there are, and panel GEMMs are mutually independent. Splitting K instead would re-read and re-write all of C per group — measured at several ms per pass at n=4096, which is most of what the pipeline is trying to hide – and would serialize the panel GEMMs through that accumulation.

Panel B-slices are double-buffered across `OZAKI_NSLOTS` slots (2 by default; 3 and 4 measured slower). A panel reusing a slot waits only on the GEMM that last read it, which is what decouples consecutive panels. Because a panel is a slice of B rather than all of B, panelling also lowers peak memory relative to the unpanelled path.

Panelling is skipped when it cannot pay: too few tiles per panel to saturate the device, `N <= tn`, K-grouping active, device-resident operands, or `OZAKI_NPANEL=1`. Requesting a B cache (`OZAKI_CACHE=2` or `3`) takes precedence — a cached slice buffer must hold all of B, so caching and panelling are alternative ways to avoid the same work (reuse across calls versus overlap within one). With `transb` a panel is a strided row block, so B is uploaded whole and only the preprocessing and download overlap.

Disabled by default because the automatic width does not account for shape: measured on PVC (fp64) it costs up to 12% on square shapes and gains up to 7% where `N` is much larger than `M`. Set `OZAKI_NPANEL=0` to enable it where `N` is the dominant dimension.

#### Kernel Registry

Both schemes compile fused GEMM kernels on demand via a JIT registry keyed by the output tile plus the bounds-checking variant; Scheme 1 additionally keys on the compile-time cutoff (OZAKI_CUTOFF), which lets the compiler eliminate dead slice-pair iterations and reduce register pressure. The first call with a given key triggers JIT compilation (~100 ms); subsequent calls hit the registry cache. A workload with a single matrix shape produces one specialization per scheme.

#### Example

```
./ozaki.x 256
```

Scheme 2 on a large matrix:

```
OZAKI=2 ./ozaki.x 4096
```

Adaptive scheme selection with caching:

```
OZAKI=3 OZAKI_CACHE=3 ./ozaki.x 4096
```

#### Quick Tuning Guide

Scheme 2 (CRT, OZAKI=2, default): fixed cost of P integer GEMMs plus hierarchical Garner reconstruction. Predictable performance regardless of data distribution. Use OZAKI_GROUPS for K-grouping at large sizes. The hierarchical CRT (OZAKI_HIER, on by default) halves private residue arrays and enables GRF128 for doubled thread occupancy. Selecting flat reconstruction takes `OZAKI_HIER=0 OZAKI_FRACCRT=0` with `OZAKI=1` or `OZAKI=2`, because the per-group fractional CRT and the adaptive scheme both need the hierarchy; the combination warns when it cannot be honoured.

Scheme 1 (mantissa slicing, OZAKI=1): up to S*(S+1)/2 integer GEMMs, but adaptive cutoff can reduce this substantially for narrow exponent spans. Use OZAKI_TRIM to trade accuracy for speed.

Adaptive (OZAKI=3): automatically picks the cheaper scheme per call based on preprocessing occupancy. Best with OZAKI_CACHE=3 to avoid repeated occupancy readbacks.

Enable OZAKI_CACHE=3 when A or B stays constant across calls. Note that only OZAKI_CACHE=0 disables the cache: a negative value is another spelling of 3.

### Small Matrix Multiplication (SMM) — OpenCL

Batched small matrix multiplications (SMM) on OpenCL devices via the ACC LIBSMM interface. Originated from the DBCSR OpenCL backend, adapted to run on top of LIBXSTREAM. Includes GPU-accelerated kernels, a benchmark driver, and an auto-tuning framework.

### Build

```
cd samples/smm
make [WITH_GPU=<device>] [ELEM_TYPE=<type>]
```

Produces `acc_bench.x`. Requires an OpenCL runtime, BLAS (`BLAS=2`), and LIBXS built from a sibling directory.

| Make variable | Default | Description |
|---|---|---|
| ELEM_TYPE | `double` | Element precision: `double` or `float` |
| WITH_GPU | auto | Device for tuned parameters (PVC, A100, ...). Fallback: all CSVs |

A CMake project (e.g., DBCSR) builds LIBSMM into its own target with `libxstream_add_smm(<target>)` after `find_package(libxstream)`, which also generates `smm_kernels.h` into the target's build tree.

### Benchmark Driver

```
./acc_bench.x [nrepeat [batchsize [M [N [K [nc [na [nb]]]]]]]]
```

The kernel shape can also be given as `MxNxK`:

```
./acc_bench.x 5 30000 13x5x7
```

The first argument can be a file with one set of parameters per line. The batchsize argument accepts K/M/G suffixes for memory-budget mode.

| Argument | Default | Description |
|---|---|---|
| nrepeat | 66 (3+CHECK) | Number of timed repetitions |
| batchsize | 30000 | Matrix products per batch (stack size) |
| M | 23 | Rows of A and C |
| N | M | Columns of B and C |
| K | M | Columns of A / rows of B |
| nc | batchsize/16 | Number of unique C-matrices |
| na | 10*nc* / *Number of unique A-matrices* | / *nb* / *10*nc | Number of unique B-matrices |

**Environment Variables (Benchmark)**

| Variable | Default | Description |
|---|---|---|
| CHECK | -1 | Accuracy: negative=1E-3 (float: 2E-3), 0=off, >0 scales |
| CHECK_H2D | - | Minimum H2D bandwidth (GB/s); fail if below |
| CHECK_DEV | - | Minimum device GFLOPS/s; fail if below |
| CHECK_HST | - | Minimum host GFLOPS/s; fail if below |
| DEVICE | 0 | Device index (multi-rank: device = rank % ndevices) |
| NREPEAT_H2D | 1 | H2D copy repetitions for bandwidth measurement |
| NREPEAT_SMM | 1 | SMM kernel launches per timed iteration (for profiling) |
| BATCHSIZE_SMM | - | Override batchsize (and optionally nrepeat: `bs,nrep`) |
| UNIFIED | 0 | 1: kernels read the stack from host memory if possible |

### LIBSMM Kernel Parameters

OpenCL kernels are generated at runtime for any (M, N, K) shape within MAX_KERNEL_DIM. Tuned parameters can be embedded at build time or loaded at runtime for better performance.

**Environment Variables (LIBSMM)** Transpose kernel:

| Variable | Default | Description |
|---|---|---|
| OPENCL_LIBSMM_TRANS_BUILDOPTS | - | Extra OpenCL build options |
| OPENCL_LIBSMM_TRANS_INPLACE | 0 | Non-zero: in-place transpose (no LDS) |
| OPENCL_LIBSMM_TRANS_BM | auto | Block size in M-direction (0 < BM <= M) |

Multiply kernel:

| Variable | Default | Description |
|---|---|---|
| OPENCL_LIBSMM_SMM_BUILDOPTS | - | Extra OpenCL build options |
| OPENCL_LIBSMM_SMM_PARAMS | auto | Tuned parameters: 0=disable, or CSV path |
| OPENCL_LIBSMM_SMM_PREDICT_CONF | 0.5 | Confidence a predicted parameter needs; below, its default is used |
| OPENCL_LIBSMM_SMM_BS | auto | Intra-kernel mini-batchsize |
| OPENCL_LIBSMM_SMM_BM | auto | Block size in M-direction (0 < BM <= M) |
| OPENCL_LIBSMM_SMM_BN | auto | Block size in N-direction (0 < BN <= N) |
| OPENCL_LIBSMM_SMM_AP | auto | Access pattern for parameter/stack array |
| OPENCL_LIBSMM_SMM_AA | auto | Access pattern for A-matrix array |
| OPENCL_LIBSMM_SMM_AB | auto | Access pattern for B-matrix array |
| OPENCL_LIBSMM_SMM_AC | auto | Access pattern for C-matrix array |

The full list of tunable parameters is available via `tune_multiply.py --help`. Some parameters produce distinct code paths (e.g., SMM_BS=1 vs SMM_BS=2), so the parameter space is non-smooth.

### Tuned Parameters

Pre-tuned parameter sets in `params/`:

| File | Device |
|---|---|
| tune_multiply_PVC.csv | Intel Data Center GPU Max (PVC) |
| tune_multiply_BMG.csv | Intel Battlemage (BMG) |
| tune_multiply_A100.csv | NVIDIA A100 |
| tune_multiply_H100.csv | NVIDIA H100 |
| tune_multiply_GH200.csv | NVIDIA GH200 |
| tune_multiply_V100.csv | NVIDIA V100 |
| tune_multiply_P100.csv | NVIDIA P100 |
| tune_multiply_Mi250.csv | AMD MI250 |

Parameters are matched by device ID at runtime with best-match fallback. Single and double precision coexist in the same CSV.

```
## Use embedded parameters (default)
./acc_bench.x 5 30000 13 5 7

## Disable tuned parameters
OPENCL_LIBSMM_SMM_PARAMS=0 ./acc_bench.x 5 30000 13 5 7

## Load from a specific CSV file
OPENCL_LIBSMM_SMM_PARAMS=params/tune_multiply_PVC.csv ./acc_bench.x 5 30000 13 5 7
```

Rebuild with a specific device's parameters embedded:

```
make realclean
make WITH_GPU=PVC
```

Show which embedded parameter set a device name selects:

```
make match
./acc_match.x "NVIDIA H200"
```

### Auto Tuning

Uses OpenTuner to explore the parameter space per (M, N, K) kernel. The tuner communicates with the benchmark driver through environment variables.

```
cd samples/smm
pip install -r requirements.txt
```

Build `acc_bench.x` before tuning. Keep GPU clocks and driver state as stable as practical during a run.

**tune_multiply.py** Use `tune_multiply.py` when you want direct control over one tuning run or over JSON maintenance.

Tune one kernel and write JSON results in the current directory:

```
./tune_multiply.py 13x5x7
```

Limit the search time, choose the JSON directory, and set the benchmark batch size:

```
mkdir -p params/local
./tune_multiply.py 13x5x7 --stop-after=300 -p params/local -s 30000
```

Tune several explicit kernels from a file, one `MxNxK` per line. A `#` starts a comment, so a line can be annotated or taken out of the list:

```
printf '%s\n' 13x5x7 23x23x23 32x32x32 > kernels.txt
./tune_multiply.py kernels.txt --stop-after=180 -p params/local
```

Merge JSON files into a CSV file (without a shape, `-p` or `--prefer` alone merges as well):

```
./tune_multiply.py -m -p params/local -o tune_multiply_local.csv
```

A JSON file is named after its kernel and a hash of its parameters, for example `tune_multiply-double-23x23x23-1a2b3c4d.json`, with `-s<N>` after the shape if the batch size is not the default 30000. Tuning that finds the same parameters again keeps one file with the higher GFLOPS/s. A merge renames files of earlier forms and keeps their modification time, which `--prefer new` relies on. Keep one directory per device: a same-named file from another device is reported and left alone.

Update the device name stored in every JSON file of a directory. Given a name, the tuner writes it as-is and runs nowhere in particular. Given no name, it asks the device how it identifies itself, which requires running on the target machine:

```
./tune_multiply.py -u "Some Device [0x1234]" -p params/local
./tune_multiply.py -u -p params/local
```

Write the JSON files a CSV file was merged from (the inverse of `-m`), which needs neither a device nor the benchmark:

```
./tune_multiply.py -M params/tune_multiply_PVC.csv -p tmp/PVC
```

The names are reproduced as a tuning session would have written them, because a JSON file is named after a hash of its parameters. A file that exists is kept rather than rewritten, so an existing directory does not lose the file dates that `--plan` orders by.

Check existing JSONs without re-tuning:

```
./tune_multiply.py -c -p params/local
```

When two JSON files describe the same kernel on the same device, `--prefer` decides which one wins the CSV: `fast` (default) keeps the higher GFLOPS/s, and `new` keeps the more recent measurement, which is what a driver or hardware change calls for. A merge lists the losing files, and `-d` removes them:

```
./tune_multiply.py -p params/local --prefer new
./tune_multiply.py -p params/local --prefer new -d
```

The first command prints `Remove N (use -d)` followed by the files the second one deletes, so a merge is also the preview.

Neither direction is inherently safer. Deleting the older file discards a configuration that was faster and may be faster again, while deleting the newer one discards the evidence of a regression.

Useful options:

| Option | Meaning |
|---|---|
| –stop-after N | Stop the search after N seconds |
| -p path | JSON directory; without a shape, merge it |
| -s size | Benchmark batch size, also called stack size |
| -a level | Tuning level: 0=all … 4=least tunables |
| -m | Merge JSON files into a CSV file |
| -M file | Write the JSON files of a CSV file (see -p) |
| -o file | CSV output file |
| -u [device] | Update JSON device names (see above) |
| -c [epsilon] | Validate JSON entries |
| –prefer fast\|new | Duplicate winner; without a shape, merge |
| -d | Delete losing duplicates during merge |

The tuner can run under MPI. It detects local MPI rank variables and uses them to select `LIBXSTREAM_DEVICE`, so ranks on the same node can use different devices. This is most useful when each rank is tuning a different kernel.

```
mpirun \
  ./tune_multiply.py 13x5x7 --stop-after=300 -p params/local : \
  ./tune_multiply.py 23x23x23 --stop-after=300 -p params/local
```

Interrupted tuning writes the best result seen so far. If you tune a different kernel with `tune_multiply.py` directly, remove any old OpenTuner database first:

```
rm -rf opentuner.db
```

The wrapper below does this cleanup automatically.

**tune_multiply.sh** Use `tune_multiply.sh` when you want to expand triplet specifications, retune a directory, or split a larger tuning job into parts.

Tune a compact Cartesian-product specification:

```
./tune_multiply.sh -t 300 4 10 15, 6 7 8, 23
```

Triplets are comma-separated groups. The command above expands to all `MxNxK` combinations with `M` in `4 10 15`, `N` in `6 7 8`, and `K=23`.

Tune an explicit list instead:

```
./tune_multiply.sh -t 300 1x1x1 2x2x2 13x5x7 23x23x23
```

Split the same work into four parts and run the second part:

```
./tune_multiply.sh -t 300 -j 4 -i 2 4 10 15, 6 7 8, 23
```

Parts also split a long job into sessions on a single device. The split points depend only on the triplet list and `-j`, so each session repeats the same command and only the part number changes:

```
PART=1  # next session: 2, then 3 and 4
./tune_multiply.sh -t 300 -j 4 -i ${PART} -p params/local 4 10 15, 6 7 8, 23
```

Keep the options that shape the list unchanged between sessions (`-r`, `-m`, `-n`, `-b`, `-f`, `-k`), since any change moves the split points. With `-u`, the list comes from the JSON directory, which stays the same as long as no new shapes are added to it between sessions.

Retune every JSON file found in a directory:

```
./tune_multiply.sh -u -p params/local -t 300
```

**Retuning by age** `--plan` lists the kernels of a JSON directory oldest first and writes that list as a file of triplets, or to standard output if no file is given. A kernel is dated by its youngest JSON file, so a kernel counts as old only once all of its files are:

```
./tune_multiply.sh --plan retune.txt -p params/local -n 40
./tune_multiply.sh -f retune.txt -p params/local -t 300
```

The first command takes the 40 kernels that were tuned least recently (`-b` takes the most recent ones instead, and `-r`, `-m`, and `-n` limit the plan as they limit any other list). The second command works the plan. Tuning does not necessarily rewrite a JSON file: finding the same parameters again with a lower GFLOPS/s keeps the file and its date, so an order taken from the directory a second time can repeat the kernels it just tuned. The plan avoids this by holding the order still, which is also what makes a plan the list to keep between sessions of a split job.

A plan records its own progress: each kernel that tuned successfully is marked `#done` and is skipped when the plan is used again, so an interrupted session resumes where it stopped and a kernel that failed stays in the plan. Marking rewrites the plan, which is why it is limited to the generated file (a handwritten list is left alone) and to a single session (`-j 1`). Delete the marks to tune the plan again:

```
sed "s/^#done //" retune.txt > retune.tmp && mv retune.tmp retune.txt
```

There is no minimum age: a plan lists every kernel of the directory, and `-n` decides how much of it to take. Taken with `--plan`, it fixes a smaller campaign, and taken with `-f`, it sizes a single session, so the same command repeated works down the plan by the same budget each time:

```
./tune_multiply.sh --plan retune.txt -p params/local
./tune_multiply.sh -f retune.txt -p params/local -n 2 -t 300
```

A complete plan has no unmarked line left, which is the point to generate the next one:

```
grep -qv "^#" retune.txt || ./tune_multiply.sh --plan retune.txt -p params/local
./tune_multiply.sh -f retune.txt -p params/local -n 2 -t 300
```

A date tells when a kernel was last improved, not when it was last tuned, and `#done` is the only record of an attempt. A kernel that retuning never improves therefore keeps its date and heads the next generated plan again.

Limit the generated work before splitting it:

```
./tune_multiply.sh -t 180 -r 4 32 -m 64 -n 100 \
  4 8 16 32, 4 8 16 32, 4 8 16 32
```

Useful options:

| Option | Meaning | ||| | -t seconds | Time limit per kernel | | -p path | Directory for JSON files | | -s size | Benchmark batch size, also called stack size | | -a level | Tuning level: 0=all ... 4=least tunables | | -u | Retune JSON files found under `-p` | | –plan [file] | Write kernels of `-p` by age, oldest first | | -d | Ask the merge step to delete losing JSONs | | –prefer new | Prefer newest duplicate (default: fastest) | | -c | Continue with the next kernel after an error | | -b | Reverse triplets, before `-n` and before parts | | -j parts | Total number of tuning parts | | -i index | Part to run, using 1-based numbering | | -r low high | Keep kernels with low$\mathbf{3 < MNK <= high}3$ | | -m extent | Keep kernels with M, N, and K no larger than this | | -n count | Keep only the first count kernels (see `-b`) | | -f file | Read MxNxK list from a file, resuming a plan | | -k id | Use a predefined triplet set |

Options the wrapper does not know are passed to `tune_multiply.py`. Such an option must carry its value as `--opt=value`, since a separate value would be read as the start of the triplet specification:

```
./tune_multiply.sh -t 300 --check=0 23x23x23
```

The tuning level defaults to `-1`, which fixes the same tunables as level 2. Levels 3 and 4 fix successively more of them.

**Tuning from a File** A text file with one `MxNxK` per line can drive the tuning session. Lines starting with `#` are comments, and inline comments after `#` are stripped. Whitespace within an entry is ignored:

```
./tune_multiply.sh -t 300 -f retune_shapes.txt -p params/local
```

Under MPI, the file entries are partitioned across ranks as usual:

```
mpirun -np 8 ./tune_multiply.sh -t 300 -f retune_shapes.txt -p params/local
```

**Bulk Tuning with MPI** Under MPI, the wrapper defaults `-j` to the MPI world size and `-i` to `rank + 1`. It also forwards a normalized local rank to `tune_multiply.py`, so the Python tuner can select a different device per local rank.

For most MPI launchers, this is enough:

```
mpirun -np 8 ./tune_multiply.sh -t 300 -p params/local \
  4 10 15, 6 7 8, 23
```

You can still spell out parts explicitly when the launcher or scheduler needs it:

```
mpirun \
  ./tune_multiply.sh -t 300 -j 4 -i 1 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 2 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 3 -p params/local \
    4 10 15, 6 7 8, 23 : \
  ./tune_multiply.sh -t 300 -j 4 -i 4 -p params/local \
    4 10 15, 6 7 8, 23 \
>out.log 2>&1
```

To retune an existing JSON directory across eight MPI ranks:

```
mpirun -np 8 ./tune_multiply.sh -u -p params/local -t 300
```

**Managing Tuned Parameters** JSON files are the working format and stay on the machine that tuned them, since `params/*/` is not tracked. CSV files are the deployable format. `smm_params.sh` converts between the two for every device at once, pairing a directory `params/<device>-0x<id>` with the CSV file `params/tune_multiply_<device>.csv`:

```
./smm_params.sh            # merge every device
./smm_params.sh "H100*"    # merge matching devices only
./smm_params.sh -e         # the reverse: write JSONs to tmp/
```

The device id in the directory name is what tells two variants of the same GPU apart. A merge compares it against the id of the parameters it merged and against the CSV file it is about to replace, and reports both ids rather than overwriting the wrong file. A device without a CSV file needs `-f`, which is how a new device gets its first one.

A typical retune flow is:

```
make realclean
make WITH_GPU=P100
./tune_multiply.sh --plan retune.txt -p params/P100-0x2f6c
./tune_multiply.sh -f retune.txt -p params/P100-0x2f6c -t 300
./smm_params.sh P100
```

Keep GPU driver state persistent during tuning (e.g., `nvidia-smi -pm ENABLED` on headless NVIDIA systems).

**Starting a New Device** Tuning from scratch starts from the benchmark's own defaults. Starting from parameters that already work on a similar GPU is usually better, so `-e` expands a CSV file back into JSON files and any published device can seed a new one. Pick the closest relative, e.g. the previous generation of the same vendor:

```
./smm_params.sh -e P100
mv tmp/P100-0x2f6c params/NEW-0x0000
```

`-e` writes below `tmp/` and never into a corpus, so it cannot reset the file dates of parameters that are already tuned. The expanded files still name the device they came from. On the target machine, let the tuner ask the device instead:

```
./tune_multiply.py -u -p params/NEW-0x0000
./smm_params.sh -f NEW
```

The merge reports the id it actually found, which is the id to put into the directory name. Rename accordingly and the pair is consistent:

```
mv params/NEW-0x0000 params/NEW-0x1234
./smm_params.sh -f NEW
```

From there, retuning is the normal flow: the seeded parameters are what the search starts from, and `--plan` keeps a record of what was retuned.

## Stencil

Finite-difference stencil computation for seismic wave propagation (RTM, FWI) on GPUs. Three kernel paths are available:

- **FP32** (default): dedicated scalar kernel with SLM blocking
- **BF16-DPAS** (STENCIL_BF16=1): Dekker-split BF16 via hardware matrix units (DPAS/XMX)
- **INT8-DPAS** (STENCIL_INT8=1): Ozaki-1 INT8 with carried-forward exponent

The DPAS paths achieve single-precision accuracy from low-precision tensor operations.

### Method

The 3D Laplacian is decomposed into three 1D operators applied along each axis (dimension splitting). Each 1D operator D is a BLK x BLK banded Toeplitz matrix (bandwidth 2R+1) representing the FD stencil.

Parameters (compile-time):

```
BLK       = 32    block side length (32^3 output cube)
RADIUS    = 4     half-order (8th-order FD, 9-point stencil)
```

**FP32 path (default)** The dedicated FP32 kernel uses SLM tiling with a sliding window along the slow axis. Work-group size 32x8, cooperative SLM fill, no DPAS dependency. Supports XYZ and ZYX memory layouts, PML absorbing boundaries, and compact/dispersion-fitted operator methods.

**BF16 path (STENCIL_BF16=1)** The wavefield block P and operator D are Dekker-split into BF16 digits. The matrix product D x P is computed as a sum of BF16 x BF16 DPAS calls that accumulate into FP32.

```
NDIGITS_A = 2     BF16 digits for operator (11-bit range)
NDIGITS_X = 3     BF16 digits for wavefield
```

Products per dimension: NDIGITS_A x NDIGITS_X = 6 DPAS calls. Isotropic total: 3 x 6 = 18 DPAS calls per output block.

**INT8 path (STENCIL_INT8=1)** Ozaki-1 slicing: the FD operator fits in a single 7-bit signed digit (NSLICES_A=1). The wavefield is sliced into 1-3 digits at runtime depending on local dynamic range.

```
NSLICES_A = 1     INT8 digits for operator
NSLICES_X = 1-3   INT8 digits for wavefield (runtime adaptive)
K_PAD_I8  = 64    K dimension (k=32 DPAS alignment)
```

Products per dimension: 1 x nslices_eff = 1-3 DPAS calls. Isotropic total: 3 x (1-3) = 3-9 DPAS calls per output block.

A per-block carried-forward exponent (exp_buf) tracks the dynamic range across time steps. Double-buffered output-based tracking avoids race conditions between neighboring blocks.

### 2D Block I/O

The kernel exploits Intel 2D block load instructions for the A-side (operator) and sub-group block reads for the B-side (wavefield in SLM):

BF16 path:

```
A: intel_sub_group_2d_block_read_16b_8r16x1c
B: intel_sub_group_2d_block_read_transform_16b_16r16x1c (VNNI)
```

INT8 path:

```
A: intel_sub_group_2d_block_read_8b_8r32x1c (k=32)
B: intel_sub_group_block_read8 from row-major SLM
```

The operator D is stored dense (banded zeros included). The structural zeros cost no memory bandwidth (D fits in L1 after first access) and no effective compute (kernel is memory-bound on wavefield reads).

The INT8 kernel gathers the wavefield directly into SLM during the fill loop (no separate preprocess kernel).

### Build

```
make DBG=1 PEDANTIC=2
```

Requires LIBXS (sibling directory ../../../libxs) and an OpenCL 2.0 runtime with cl_intel_subgroup_2d_block_io and DPAS support.

Build-time defines go into ECFLAGS, e.g. `make ECFLAGS="-DSTENCIL_PML_W=27"` (DFLAGS and CFLAGS are assembled by the build and a command line assignment would replace them):

```
STENCIL_PML_W     absorbing layer width in cells (default: 20).  A
                  caller supplying its own eta sets its own damping
                  width here; a larger value is safe, a smaller one
                  leaves damped cells unattenuated.
```

```
make OCL=0
```

Compiles the FP32 kernel (kernels/stencil_fp32.cl) with the C compiler and runs it on the CPU instead of the GPU. Needs neither OpenCL nor the LIBXSTREAM library. The kernel is specialized at build time, so the work-group shape and optionally the grid extent are pinned by CPUDEF:

```
make OCL=0 CPUDEF="-DWG_X=128 -DWG_Y=8"
```

WG_X defaults to the full fast axis (capacity 1024) and WG_Y to 8. Bind the threads, which is what `make OCL=0 test` does:

```
OMP_PROC_BIND=spread OMP_PLACES=cores ./stencil.x -n 512
```

The kernel is compiled once per combination the run may ask for, and `stencil_configure` selects between them, which is how these environment variables keep working without a JIT:

```
STENCIL_BF16S    wavefield storage limbs (1 = single BF16, 2 = two)
STENCIL_FP16S    IEEE FP16 wavefield storage (0/1)
STENCIL_HALO     halo padding size per axis
STENCIL_METHOD   operator method (0-3)
STENCIL_LAYOUT   accepted when it names the layout built in
STENCIL_PML      accepted when it matches the build
```

Four storage formats times four operator radii times the two boundary treatments (clamp the gather, or read it out of the halo, which the grid decides as on the device) are 32 instances, about 260 KB of code. Drop the compact radii for a build of eight:

```
make OCL=0 CPUDEF="-DSTENCIL_CPU_COMPACT=0"
```

Asking for a combination the build does not carry is an error naming the flag to rebuild without, never a silent substitution. `STENCIL_PADDED` forces the boundary treatment and then compiles only that half.

The benchmark driver reaches the host build through `--exe`, which makes the CPU kernel sweepable like any device variant:

```
make OCL=0 BLDDIR=/tmp/b/obj OUTDIR=/tmp/b
./stencil.py --exe /tmp/b/stencil.x --kernel bf16s --sizes 256,512
```

BF16S costs about 8% against FP32 and FP16S about 3x, because the FP16 conversion does not vectorize in the C89 mode `PEDANTIC=2` selects. BF16S=2 (`--kernel bf16s-split`) holds two limbs and therefore moves as many bytes as FP32: it buys precision, not bandwidth.

## Usage

```
./stencil.x [options]

-n <N>          grid dimension (NxNxN, default 256)
-nx/ny/nz <N>   individual grid dimensions
-t <steps>      time steps (default 100)
-d <dims>       operator terms: 3=isotropic, 9=TTI (default 3)
-h <spacing>    grid spacing in meters (default 10.0)
-v <model>      velocity: const | grad | layered | <file.bin>
-vmin <vel>     minimum velocity m/s (default 1500)
-vmax <vel>     maximum velocity m/s (default 4500)
-f <freq>       source frequency Hz (default 25)
-w <steps>      warmup steps excluded from timing (default 5)

-seg-salt       preset: SEG/EAGE Salt (676x676x210, h=20m)
-overthrust     preset: SEG/EAGE Overthrust (801x801x187, h=25m)
```

Performance is reported as GPoints/s (grid points updated per second).

### Velocity Models

The driver supports loading external velocity models as flat binary files (float32, x-fastest order). Several standard models are publicly available for benchmarking.

```
Grid:    676 x 676 x 210   (n1=210 depth, n2=n3=676 lateral)
Spacing: 20 m
Range:   1500 - 4482 m/s
Size:    ~385 MB (float32)
Notes:   Salt body with sharp velocity contrasts.
         Standard RTM benchmark (Aminzadeh et al., 1997).
```

Original (20 m): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/fvp

Resampled (40 m, 115x338x338, for cheaper runs): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/BENCH_27PT/v.bin

Reference wavefield (frequency-domain CBS solution): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/BENCH_27PT/cbs_salt_real.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/BENCH_27PT/cbs_salt_imag.bin

```
Grid:    801 x 801 x 187   (n1=187 depth, n2=n3=801 lateral)
Spacing: 25 m
Range:   2179 - 6000 m/s
Size:    ~450 MB (float32)
Notes:   Layered geology with thrust faults.  Smoother than Salt.
         Reference: Aminzadeh, Brac, and Kunz (1997).
```

Original (25 m): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp

Resampled (50 m, 94x401x401): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp.r50

2D section (slice i3=455, 187x801): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/overthrust2D

Reference wavefield (CBS, 50 m grid): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/BENCH_27PT/v.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/BENCH_27PT/cbs_overthrust_real.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/BENCH_27PT/cbs_overthrust_imag.bin

```
Grid:    2301 x 751         (n1=751 depth, n2=2301 lateral)
Spacing: 4 m
Notes:   Classic 2D migration benchmark (Versteeg, 1994).
```

Velocity (vp.bin): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Marmousi/vp.bin

Density (rho.bin): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Marmousi/rho.bin

```
Grid:    1911 x 12601
Spacing: 6.25 m
Notes:   Challenging sub-salt imaging (Billette and Brandsberg-Dahl, 2004).
         Available from SEG open data.
```

SEG wiki page (registration may be required): https://wiki.seg.org/wiki/2004_BP_Velocity_Estimation_Benchmark_Model

```
Grid:    641 x 209          (n1=209 depth, n2=641 lateral)
Spacing: 25 m
Type:    Acoustic VTI (vertical transverse isotropy)
Fields:  Vp, epsilon, delta, eta, Vnmo, density
Notes:   Representative of North Sea environments.
         Exercises anisotropic operator (pure terms with
         modified coefficients; cross-terms vanish for VTI).
```

Download (Geoazur WIND): https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Valhall2D/vp_true.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Valhall2D/epsilon_true.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Valhall2D/delta_true.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Valhall2D/eta_true.bin https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Valhall2D/rho_true.bin

```
Grid:    ~1911 x 12480
Spacing: 6.25 m
Type:    Acoustic TTI (tilted transverse isotropy)
Fields:  Vp, epsilon, delta, theta (tilt angle)
Notes:   Canonical TTI benchmark (Shafiq et al., 2007).
         Exercises full cross-derivative kernel with non-zero
         tilt angles.  This is the standard reference for TTI
         stencil performance.
```

SEG wiki page (registration required): https://wiki.seg.org/wiki/2007_BP_Anisotropic_Velocity_Benchmark_Model

**Synthetic 3D TTI (from Overthrust)** No public 3D TTI model exists. The standard practice in GPU stencil papers is to overlay synthetic Thomsen parameters on the 3D Overthrust velocity model:

```
Vp:      from Overthrust (801x801x187, 25 m)
epsilon: constant 0.1 - 0.2 (or linear gradient)
delta:   constant 0.05 - 0.1
theta:   smooth tilt (e.g., 20-45 degrees, linearly varying with depth)
phi:     azimuthal angle (0 or smooth variation)
```

This gives a realistic velocity structure with controlled anisotropy parameters for benchmarking the TTI kernel (-d 9 mode).

**Data hosting (Geoazur WIND project)** All models above (except BP 2004) are hosted by the WIND project at Universite Cote d'Azur / Geoazur. Index page:

```
https://www.geoazur.fr/WIND/bin/view/Main/Data/
```

**Usage with this benchmark** Download the velocity binary and run:

```
wget https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Salt/fvp
./stencil.x -seg-salt -v fvp -t 1000

wget https://www.geoazur.fr/WIND/pub/nfs/FWI-DATA/GEOMODELS/Overthrust/fvp
./stencil.x -overthrust -v fvp -t 1000
```

The binary format is flat float32 with n1 (depth) as the fastest dimension. Velocities are in m/s; the driver squares them internally. Byte order is native (little-endian on x86).

### Performance Metric

The standard metric for stencil codes is GPoints/s:

```
GPoints/s = (nx x ny x nz x nsteps) / (time x 1e9)
```

This measures throughput independent of the internal algorithm (direct FD, GEMM-based, FFT, etc.) and is directly comparable across:

```
- Devito (Imperial College) --- symbolic PDE -> optimized C/OpenCL
- minimod (TotalEnergies) --- open-source GPU stencil mini-app
- Published results on PVC, A770, H100, MI300X
```

For reference, a memory-bandwidth-bound 8th-order stencil on PVC (HBM bandwidth ~3.2 TB/s) achieves approximately 200-400 GPoints/s depending on grid size and occupancy.

### TTI (Anisotropic) Mode

With -d 9, the kernel computes the full tilted transverse isotropy operator with cross-derivative terms. Each cross-term uses SLM to buffer the intermediate result between two GEMM phases:

```
T  = D_j x P          (first GEMM)
T  = c_ij . T         (point-wise anisotropy scaling)
Y += D_i x T          (second GEMM)
```

SLM budget per cross-term: 2 x K_PAD x XMX_N x sizeof(ushort), rounded up to the DPAS K block size. Total for 3 cross-terms with barrier: fits within 128 KB (Xe2/BMG).

### Operator Paths

The standard direct kernel applies one wide-reach operator (radius-4, 9-point) per dimension. The compact variants are independent runtime paths: they compile the same Ozaki-split BF16 x BF16 DPAS primitive with a smaller gather radius and rely on time evolution to build longer effective reach.

Methods (selected via STENCIL_METHOD environment variable):

0 = direct K=1, r=4 standard high-order (default) 1 = compact-r1 K=4, r=1 compact tridiagonal runtime path 2 = compact-r2 K=2, r=2 compact pentadiagonal runtime path 3 = compact-fit K=4, r=1 placeholder for dispersion-fitted coefficients

The "compact-fit" mode uses the free degrees of freedom in the K-factor coefficient product to match or exceed the dispersion quality of the standard 8th-order stencil at target frequencies, while retaining the memory savings of the cascade. Coefficients are precomputed once at initialization for the grid's frequency content.

The first compact path is implemented for the isotropic apply kernel in methods 1-3. TTI cross-terms still use the direct two-phase DPAS path.

Environment variables controlling kernel specialization:

```
STENCIL_BF16      BF16-DPAS kernel (1=native BF16, 2=FP32-split via BF16)
STENCIL_BF16S     BF16 wavefield storage limbs (0=off, 1=single 2-byte
                  limb halves traffic, 2=two limbs FP32-equivalent)
STENCIL_FP16S     IEEE FP16 wavefield storage (0/1); single 2-byte value
                  per point, halves traffic (10-bit mantissa, FP32 kernel)
STENCIL_INT8      INT8-DPAS kernel (1=native INT8, 2=FP32-split via INT8)
STENCIL_METHOD    operator method (0-3, default 0)
STENCIL_STRIPS_PER_WG
     adjacent N-strips handled by one work-group (default: 2)
STENCIL_SG        subgroup size override (default: device preferred)
STENCIL_GRF256    force 256-GRF mode (0/1, default: auto)
STENCIL_TRIM      drop least-significant digit products (BF16 path)
STENCIL_LU        loop unroll strategy (-1=none, 0=inner, 1=outer)
STENCIL_LAYOUT    memory layout (0=XYZ, 1=blocked, 2=ZYX)
STENCIL_HALO      halo/padding size per axis
STENCIL_PML       enable PML absorbing boundary (0/1)
STENCIL_FP32_WG_X, STENCIL_FP32_WG_Y
   FP32 direct-kernel work-group shape (default: 32x8)
```

```
    STENCIL_FP32_SBLOCK
      FP32 direct-kernel slow-axis SLM microblock (1 or 2,
      default: 2 except ZYX+PML)
STENCIL_FP32_BLOCK_IO
   enable FP32 Intel 2D block reads for padded 32x8 cases (0/1, default: 0)
```

Specialized kernels are compiled on first use and cached in a thread-safe registry keyed by the method, compact-step parameters, strip grouping, subgroup size, packed specialization flags, FP32 work-group shape, grid shape, and term count. Subsequent launches with the same parameters are zero-cost.

Future extension: per-block adaptive method selection (different K in regions with different velocity/frequency content). This requires per-block dispatch logic and is not yet implemented.

### Integration

The stencil API (stencil_opencl.h) accepts device pointers for all buffer arguments (wavefield, output, velocity). No host-device transfers happen inside the dispatch path — data stays on-device across the full time-stepping loop.

**Initialization and USM control** LIBXSTREAM provides libxstream_init_config for explicit control over the memory model before initialization:

```
libxstream_init_config_t cfg;
libxstream_init_config_default(&cfg);  /* all fields = -1 (default) */
cfg.usm = 1;      /* force Intel USM extensions */
cfg.device = 0;   /* select device index */
libxstream_init_config(&cfg);
```

USM levels (cfg.usm):

```
-1  env/default (LIBXSTREAM_USM, or SVM coarse-grain fallback)
 0  disable USM, force clCreateBuffer path
 1  Intel USM extensions (clDeviceMemAllocINTEL)
 2  OpenCL 2.0 SVM coarse-grain only
 3  OpenCL 2.0 SVM with device-reported capabilities
```

When USM is active (level 1), device pointers from any source (SYCL, Level Zero, raw clDeviceMemAllocINTEL) can be passed to the stencil API without conversion.

**SYCL interoperability** SYCL USM device allocations (sycl::malloc_device) can be passed directly to stencil_apply_laplacian on Intel GPUs. The underlying mechanism is clSetKernelArgMemPointerINTEL, which accepts the same raw pointers that SYCL produces via the Level Zero unified runtime.

Requirements:

```
- Initialize LIBXSTREAM with cfg.usm = 1, or set the
  environment variable LIBXSTREAM_USM=1.
- The SYCL queue and the libxstream OpenCL context must target
  the same physical device (shared Level Zero context).
- Synchronization: use sycl::queue::ext_oneapi_submit_barrier()
  before calling stencil_apply_laplacian, and
  libxstream_stream_sync after, to order work correctly.
- No format conversion needed: float32 USM buffers are used
  as-is (velocity is expected as v^2).
```

**External OpenCL consumers** Any OpenCL application can call the API by passing cl_mem-backed device pointers obtained via libxstream_mem_allocate, or USM pointers from clDeviceMemAllocINTEL directly. The dispatch layer auto-detects USM availability and selects the appropriate kernel argument-setting path.

**Scalar proxy (non-DPAS devices)** The kernels include a scalar fallback that runs on any OpenCL 1.2+ device without DPAS/XMX hardware. This enables functional testing and correctness verification on iGPUs, CPUs, and other devices. Performance is not representative — the proxy exists purely for development and validation.

## File Layout

```
samples/stencil/
  Makefile              build rules
  README.md             this file
  stencil.c             host driver (benchmark, model loading)
  stencil.py            Python benchmark harness (CSV/plot output)
  stencil_opencl.c      OpenCL context, kernel dispatch, exp_buf management
  stencil_opencl.h      public API, compile-time parameters
  stencil_kernels.h     generated at build time from .cl sources
  kernels/
    stencil_common.cl   parameters, gather macros, layout indexing
    stencil_fp32.cl     FP32 path: stencil_apply_direct (default)
    stencil_bf16.cl     BF16 path: stencil_apply, stencil_apply_tti
    stencil_int8.cl     INT8 path: stencil_apply_int8

libxstream/opencl/
  libxstream_common.h   IEEE utilities, BF16 conversion, EXP2I, unroll macros
  libxstream_ozaki.h    Dekker split, BF16 DPAS macros
```